

React Fundamentals

Full Stack Open - Part 1a Notes

1. Traditional vs. Modern Web Apps

Traditional Web Pages: Behave like a constant loop of POST/GET requests. Every action forces the server to reload the entire page, sending HTML, CSS, and JS over and over. This is slow and inefficient.

Modern Single Page Apps (SPA): Solves this by fetching the HTML/CSS/JS exactly *once*. After that, React acts as the engine, rendering components dynamically on the client-side and only talking to the server to send/receive raw JSON data.

2. Initializing React with Vite

To write React, you need background tools (local server, compiler, etc.). Setting this up manually is a nightmare. **Vite (pronounced "veet")** handles all this setup automatically.

The Setup Command:

```
npm create vite@latest
```

- **Project name:** `part1`
- **Framework:** `React`
- **Variant:** `JavaScript`
- **Lint:** `ESLint` (The industry standard for catching errors early)

Starting the Server:

Vite might ask *"Install with npm and start now?"* If you say no, you must run these manually:

```
cd "folder-name"
npm install      // Downloads React and Vite into node_modules
npm run dev      // Starts the local development server
```

Cleanup Rule: For plain React apps, delete `App.css`, `index.css`, and the `assets` folder generated by Vite.

3. Components & Architecture

What is a Component? It is simply a JavaScript function that returns JSX (HTML). Think of it as a custom HTML tag you invent (e.g., `<Header />`). It makes code reusable and organized like Lego blocks.

The Core Files:

- **App.jsx:** This is your main component. All content to be rendered goes here or inside components imported here.
- **main.jsx:** The "middle-man". You do not edit this. It imports **App.jsx** and injects it into the actual **index.html** file using `createRoot(...).render(...)`.

4. JSX (JavaScript XML)

JSX is HTML written directly inside a JavaScript file. It lets you inject dynamic JS variables using curly braces `{}`.

The Secret of JSX: Browsers do not understand JSX. If sent directly to Chrome, it would crash. Behind the scenes, the compiler turns your beautiful JSX into ugly JavaScript.

What you write (JSX):

```
const App = () => {  
  return (  
    <div>  
      <p>Hello</p>  
    </div>  
  )  
}
```

What the browser actually sees:

```
const App = () => {  
  return React.createElement(  
    'div',  
    null,  
    React.createElement('p', null,  
      'Hello')  
  )  
}
```

Why use it? It completely removes the need for painful DOM manipulation (like `document.createElement`) while keeping the massive power of JavaScript.

5. Props (Passing Data)

Props allow you to pass dynamic data from a parent component down to a child component.

- **The Unwritten Rule:** Always name the parameter **props**. Never name it **data** or **heading**.
- **Naming Convention:** Always use **camelCase** (e.g., `userName`, `userAge`).

```
// 1. Defining the Component to receive props  
const Hello = (props) => {  
  return (  
    <div>  
      <p>Hello {props.name}, you are {props.age} years old!</p>  
    </div>  
  )  
}
```

```
    </div>
  )
};

// 2. Passing the data from the Parent
const App = () => {
  let userName = "Vinayak";
  return (
    <div>
      <Hello name={userName} age={23} />    <!-- Passing variable and number -->
      <Hello name="Varoon" age={25} />      <!-- Passing hardcoded string -->
    </div>
  )
};
```

Prop Drilling: The architecture of passing props from `App` → `Content` → `Part`. This is how complex data moves through a React application.

6. Golden Rules & Common Errors

The 3 Golden Rules of Components

1. **Capitalize Component Names:** If you name it `hello`, React thinks it's a regular HTML tag. It must be `Hello`.
2. **One Root Element:** A component can only return ONE parent wrapper. You cannot return two elements side-by-side. Wrap them in a `<div>` or an empty Fragment `<> ... </>`.
3. **Never Nest Definitions:** Never define a component inside another component's curly braces. Keep `const App` and `const Hello` completely separate.

The "Object rendering" Error

Error: Objects are not valid as a React child (found: object with keys {name, age}).
If you meant to render a collection of children, use an array instead.

Why it happens: You tried to print an entire JavaScript Object inside curly braces `{}` in your JSX. React doesn't know how to draw an object.

The Fix: Never render an object directly. Always drill down to the specific primitive value (string or number) you want to show, like `{props.name}`.

The ESLint Prop-Types Warning

ESLint throws a warning if you accept props without explicitly defining their data types. Since type-checking isn't needed until later in the course, silence it by adding this to the `rules:` section of

eslint.config.js:

```
"react/prop-types": 0
```

Generated for Full Stack Open - Strict Mentorship Session